

1. Datos generales

Curso	Inteligencia Artificial Aplicada al Control (ICA636-0001)
Trabajo	Tarea 4 — Control de Sistemas Dinámicos
Alumno	Joel Arzapalo Guerra
Código	20032006
Fecha	8 de julio de 2026

2. Introducción

Este trabajo analiza un conjunto de programas MATLAB que entrenan una red neuronal para actuar como **controlador** (neurocontrolador) de un sistema dinámico. El controlador recibe el estado (o su error respecto de una referencia) y produce la señal de control u que se aplica a la planta. La planta común es un sistema discreto **bilineal** de 2 estados:

$$x(k+1) = Ax(k) + Bu(k) + (Gx(k))u(k) [+W \text{ pert}],$$

donde el término $(Gx)u$ es una ganancia de entrada dependiente del estado.

El código implementa las dos estrategias de aprendizaje de la Clase 9, que se distinguen por *cómo se calcula el gradiente del costo* a través de la dinámica de la planta:

- **Aprendizaje estático** (DynamicBPControlEstaticoModelo0/1/3.m): el gradiente usa solo la sensibilidad *instantánea* del estado siguiente al control, $dxdu = B + G \cdot x$. No considera la dinámica de lazo cerrado; es simple y adecuado para **plantas estables** (mejorar el transitorio, regulación y seguimiento).
- **Aprendizaje dinámico / BPTT** (DynamicBPControl2/2a/2b.m): el gradiente se propaga *recursivamente en el tiempo* a través del jacobiano del lazo cerrado. Es más costoso pero necesario para **plantas inestables**.

El costo minimizado es $J = \frac{1}{2} \sum_k (x_k - x_k^*)^\top Q (x_k - x_k^*) + Ru_k^2$, con $Q = \text{diag}(q_1, q_2)$ y R el peso del esfuerzo de control.

El informe cubre (i) el **análisis del código** de los seis scripts y (ii) los **resultados experimentales**, entrenando los controladores y estudiando la influencia de los parámetros (peso R , número de neuronas, grado de inestabilidad, realimentación parcial, perturbaciones externas y ruido de medición). Los scripts originales (interactivos, con `input()` y `load` de pesos) se adaptaron a versiones no interactivas (`rng` fijo, hiperparámetros en código), encapsulando el entrenamiento en las funciones `train_static_ctrl.m` y `train_bptt_ctrl.m`, fieles a la matemática de los originales.

3. Análisis del código

3.1. Estructura común

Todos los scripts comparten el mismo esqueleto por celdas `%%`: definición de la planta (A, B, G, W), condiciones iniciales `x_ini` (una columna por caso, el código deduce `nini=size(x_ini,2)`), arquitectura de la red (`ne` entradas, `nm=50` neuronas, salida escalar u), carga de pesos, bucle de entrenamiento y visualización (una figura por condición inicial). La activación oculta es la sigmoide bipolar $n = 2/(1 + e^{-(m-c)/a}) - 1$ con $m = v^\top \text{in_red}$, y el control es $u = w^\top n + u^*$ (con u^* la *acción anticipativa* de equilibrio).

3.2. Familia estática: ...EstaticoModelo0/1/3.m

Modelo0 (único que *entrena*) regula al origen una planta *estable*. La clave es que el gradiente comparte un mismo coeficiente escalar por paso:

```
1 dxdu = B + G*x; % sensibilidad d x(k+1)/d u
2 er = (x - out_des); % error de seguimiento
3 erq = q .* er;
4 g = erq'*dxdu + R*u(k,j); % coeficiente escalar retropropagado
5 dJdw_t = dJdw_t + g*dudw_s; % mismo g para w, v, c, a
6 dJdv_t = dJdv_t + g*dudv_s;
```

Listing 1: Gradiente estático: solo el término instantáneo dxdu.

Incorpora **momento** ($\alpha=0.95$) y guarda los *mejores* pesos vistos (el costo puede oscilar con η alto). **Modelo1** y **Modelo3** *no* entrenan: cargan **redcontrolestatico0** y validan el mismo controlador para *seguimiento* de una consigna con *acción anticipativa* ($\text{in_red}=\mathbf{x}-\mathbf{r}$, $\mathbf{u}=\text{red}+\mathbf{uast}$) y para *robustez* ante variaciones de la planta (nominal, A menos estable, B a la mitad, G duplicada), respectivamente.

3.3. Familia dinámica (BPTT): DynamicBPControl2/2a/2b.m

Control2 estabiliza una planta *inestable* (A con diagonal > 1). El gradiente es retropropagación en el tiempo: se mantienen acumuladores recursivos de las sensibilidades del estado a los parámetros, actualizados con el jacobiano del lazo cerrado:

```
1 jacob = A + u(k,j).*G; % jacobiano de la planta d x(k+1)/d x
2 dxdu = B + G*x; % d x(k+1)/d u
3 dudx = w'*dndm*v'; % d u/d x (sensibilidad del control al estado)
4 jacob_t = dxdu*dudx + jacob; % jacobiano del lazo cerrado
5 Sw_t = dudw_s*dxdu' + Sw_t*jacob_t'; % S_p(k+1)=du/dp*dxdu' + S_p(k)*jacob_t'
```

Listing 2: Recursión BPTT vectorizada (jacobiano del lazo cerrado).

La recursión está **vectorizada** (matrices $\mathbf{Sw}_t$, $\mathbf{Sv}_t$, ... indexadas por estado) y se actualiza de forma *simultánea* para todos los estados. **Control2a** usa realimentación *parcial*: la entrada de la red es una salida medida $y = Cx$ ($\text{ne}=1$); documenta que $C = [0 \ 1]$ (medir x_2) controla, pero $C = [1 \ 0]$ (medir x_1) no. **Control2b** valida el rechazo de *perturbaciones* (consigna activa $x_1^* = 1$, perturbación senoidal por $W = [0; 1]$, y *ruido de medición* $0,005\mathcal{N}$ sumado a in_red).

Nota sobre la realimentación parcial. En la versión no interactiva, la sensibilidad del control al estado se propaga correctamente incluyendo la matriz de salida, $\text{dudx} = (\mathbf{w}' * \text{dndm} * \mathbf{v}') * \mathbf{C}$, que para $C = I$ (estado completo) recupera exactamente la expresión de **Control2**.

4. Resultados experimentales

Resultados con las versiones no interactivas (semilla fija). Las figuras se generan con **Informe/exp/run_all.m**.

4.1. Control estático: regulación en planta estable

Se entrenó el neurocontrolador estático ($n_m = 50$, $\eta = 2$, momento 0,95) para regular al origen la planta estable, partiendo de 4 condiciones iniciales. El costo converge (Fig. 1) y el lazo cerrado lleva ambos estados a cero con una señal de control suave (Fig. 2); el error final es prácticamente nulo ($\sim 10^{-13}$) en las 4 condiciones. Frente al sistema *sin* control, el neurocontrolador **acelera notablemente el transitorio** (Fig. 3), que es justamente el objetivo de mejora de respuesta del PDF.

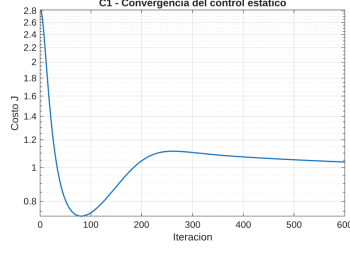


Figura 1: Convergencia del costo.

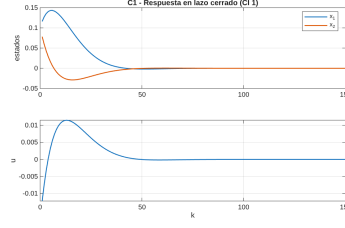


Figura 2: Respuesta en lazo cerrado.

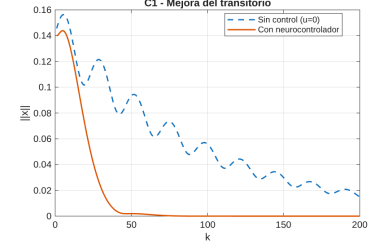


Figura 3: Con vs sin control.

Peso R , neuronas y generalización. Como la planta es estable y fácil de regular, el error final permanece despreciable para todo R ensayado (Fig. 4); el peso R solo influye levemente en el esfuerzo de control ($\Sigma u^2 \approx 0,0021$, subiendo a 0,0042 con $R = 1$). El número de neuronas tiene efecto modesto sobre el costo (Fig. 5). Lo más destacable es la **generalización**: entrenado con condiciones iniciales pequeñas, el controlador regula igualmente bien condiciones **hasta** $20\times$ **mayores**, con error final $\sim 10^{-12}$ (Fig. 6).

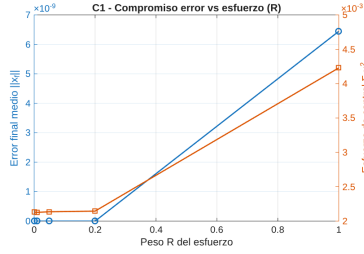


Figura 4: Compromiso error/esfuerzo vs R .

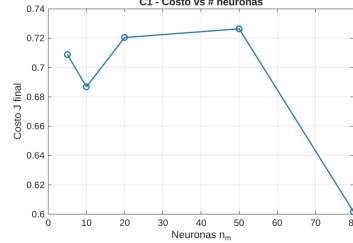


Figura 5: Costo vs n_m .

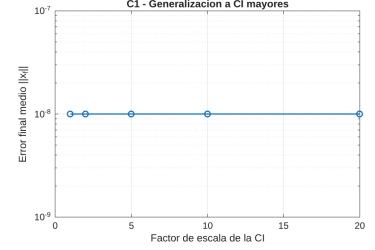


Figura 6: Generalización a CI mayores.

4.2. Robustez y rechazo de perturbaciones (estático)

Se aplicó el *mismo* controlador entrenado en la planta nominal a cuatro variantes de planta (Modelo3): nominal, A menos estable, ganancia B reducida al 50 % y término bilineal G duplicado. En los cuatro casos el lazo cerrado **sigue estabilizando** (Fig. 7, todas las trayectorias $\|x\| \rightarrow 0$): el neurocontrolador es robusto a estas variaciones sin reentrenar.

Para el rechazo de perturbaciones (Modelo1, seguimiento de $x_1^* = 0,1$ con *acción anticipativa*), se inyectó una perturbación senoidal externa de amplitud creciente. El error de seguimiento en régimen permanente crece de forma **proporcional** a la amplitud (Tabla 1, Fig. 8): el controlador *atenúa* la perturbación pero, al ser un control estático sin acción integral, no la cancela por completo.

Cuadro 1: Error de seguimiento en régimen permanente vs amplitud de la perturbación.

Amplitud perturbación	0	0.02	0.10	0.50
Error r.p. $\ x - r\ $	$1,3 \times 10^{-5}$	0.0019	0.0093	0.046

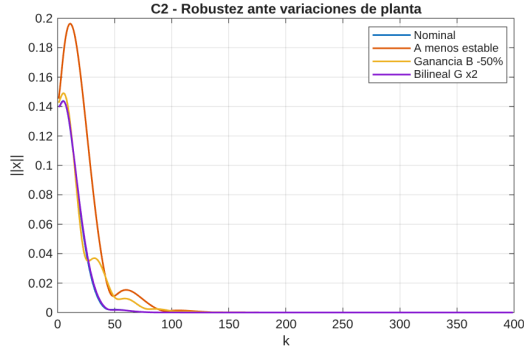


Figura 7: Robustez ante variaciones de planta.

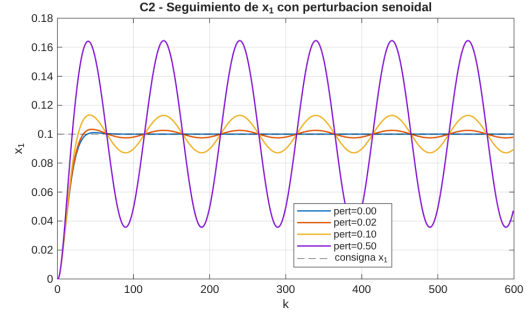


Figura 8: Seguimiento con perturbación senoidal.

4.3. Estático vs dinámico en planta inestable

Este es el experimento central. Se barrió la inestabilidad de la planta (diagonal de A de 0,98 a 1,20) y, en cada nivel, se entrenó un controlador **estático** (desde cero) y uno **dinámico BPTT** (incremental, con arranque desde pesos previos del nivel previo). Se mide el error final medio $\|x_f\|$: valores $\gg 0,1$ indican que el lazo cerrado *no* se estabiliza.

El resultado (Tabla 2, Fig. 9) es contundente: el control estático solo funciona mientras la planta es estable (0,98); **en cuanto la planta se vuelve inestable ($\geq 1,02$), diverge** (errores de 10^3 a 10^{23}), porque su gradiente ignora la dinámica de lazo cerrado. El control dinámico BPTT, en cambio, **estabiliza todos los niveles** hasta $A_{ii} = 1,20$ con error $\sim 10^{-10}$. La Fig. 10 muestra la respuesta en la planta más inestable: el estado del sistema con control estático explota mientras el BPTT lo lleva al origen. Esto confirma la tesis de la Clase 9: el aprendizaje estático no sirve para plantas inestables; se requiere el dinámico.

Cuadro 2: Error final $\|x_f\|$ según el grado de inestabilidad (diagonal de A).

Diagonal de A	0.98	1.02	1.06	1.10	1.15	1.20
Estático	7×10^{-12}	$1,9 \times 10^3$	7×10^9	5×10^{15}	$1,5 \times 10^{23}$	$1,1 \times 10^3$
Dinámico (BPTT)	3×10^{-11}	3×10^{-10}	2×10^{-10}	7×10^{-11}	5×10^{-11}	9×10^{-12}

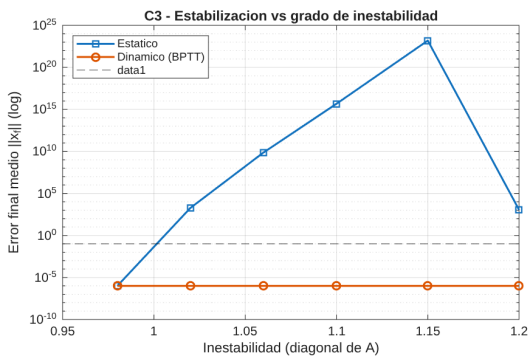


Figura 9: Estabilización vs grado de inestabilidad.

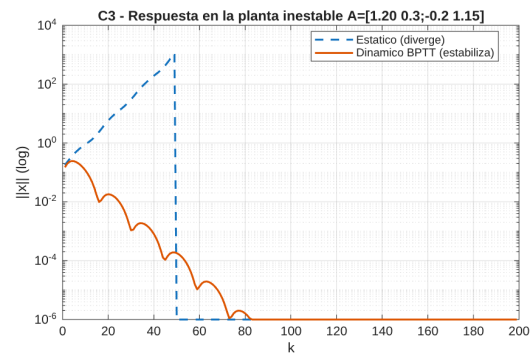


Figura 10: Respuesta en $A_{ii} = 1,20$ (escala log).

4.4. Control dinámico BPTT: realimentación parcial, perturbaciones y ruido

Todos los controladores BPTT de esta sección se entrenaron mediante **entrenamiento por etapas** (aprendizaje incremental subiendo la inestabilidad con arranque desde pesos previos). En

efecto, un hallazgo del trabajo es que entrenar BPTT *desde cero* sobre una planta muy inestable no converge; el entrenamiento por etapas —tal como emplean los scripts originales (`load incremental`)— es esencial.

Realimentación completa vs parcial. Con realimentación **completa** del estado, el BPTT estabiliza la planta inestable con error $\sim 10^{-15}$ (Fig. 11). Con realimentación **parcial** (una sola salida medida $y = Cx$, $ne = 1$), el problema es mucho más difícil: entrenando desde cero (con entrenamiento por etapas), *ninguno* de los controladores de una sola salida estabilizó la planta inestable dentro del presupuesto de entrenamiento, y su respuesta diverge. Esto es coherente con `Control2a`, que sí logra estabilizar midiendo x_2 ($C = [0 \ 1]$) pero *solo* partiendo de pesos preentrenados (*arranque desde pesos previos* extenso, `load redcontrol2a`) y descarta medir x_1 ($C = [1 \ 0]$). La conclusión práctica es doble: reducir la información medida degrada drásticamente la estabilización por realimentación de salida, y su entrenamiento exige un buen punto de partida.

Rechazo de perturbaciones. El regulador BPTT sobre la planta más inestable ($A_{ii} = 1,20$) mantiene el estado en el origen; al inyectar una perturbación senoidal externa de amplitud creciente, el error en régimen permanente crece de forma proporcional (Tabla 3, Fig. 12): sin perturbación regula de forma exacta ($\sim 10^{-15}$) y con amplitud 0,1 el error se mantiene acotado ($\approx 0,58$), sin desestabilizarse.

Ruido de medición. Entrenando con ruido de medición sumado a la entrada de la red (como en `Control2b`), el controlador resultante regula igual de bien que sin ruido: el error final permanece en $\sim 2 \times 10^{-14}$ para todos los niveles de ruido ensayados (Fig. 13). El BPTT es **robusto al ruido de medición** durante el entrenamiento, pues el gradiente promediado sobre el horizonte y las condiciones iniciales atenúa su efecto.

Cuadro 3: Error en régimen permanente del regulador BPTT vs amplitud de la perturbación.

Amplitud perturbación	0	0.01	0.03	0.05	0.10
Error r.p. $\ x\ $	10^{-15}	0.055	0.166	0.278	0.577

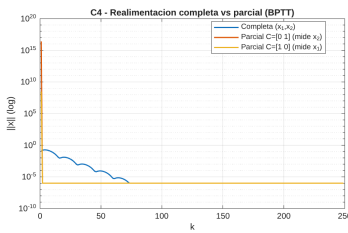


Figura 11: Completa vs parcial.

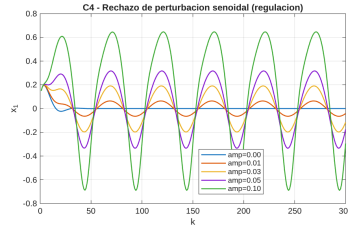


Figura 12: Rechazo de perturbación.

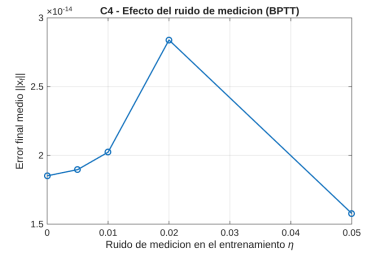


Figura 13: Efecto del ruido.

5. Conclusiones

1. **El aprendizaje estático basta para plantas estables** (Sec. 4.1): el neurocontrolador regula al origen con error despreciable, mejora fuertemente el transitorio frente al sistema sin control y *generaliza* a condiciones iniciales hasta $20\times$ mayores que las de entrenamiento.
2. **El controlador estático es robusto y atenúa perturbaciones** (Sec. 4.2): estabiliza variantes de la planta (nominal, menos estable, B a la mitad, G duplicada) sin reentrenar, y el error de seguimiento crece de forma proporcional a la amplitud de la perturbación, aunque —al no tener acción integral— no la cancela por completo.

3. **Para plantas inestables el aprendizaje estático falla y el dinámico es imprescindible** (Sec. 4.3): apenas la planta se vuelve inestable el control estático diverge, mientras que el BPTT la estabiliza en todo el rango (A_{ii} hasta 1,20) con error $\sim 10^{-10}$. Es el resultado central y confirma la tesis de la Clase 9.
4. **El entrenamiento BPTT requiere un entrenamiento por etapas** (aprendizaje incremental de la inestabilidad con arranque desde pesos previos): entrenarlo de golpe sobre una planta muy inestable no converge.
5. **La realimentación parcial es mucho más exigente** (Sec. 4.4): con el estado completo el BPTT estabiliza fácilmente, pero con una sola salida medida el entrenamiento desde cero no lo logra y requiere un buen punto de partida (*arranque desde pesos previos*), como hace el script original. Además, el BPTT rechaza perturbaciones acotadas y es *robusto al ruido de medición* durante el entrenamiento.

A. Archivos del proyecto

Los experimentos se ejecutan con `Informe/exp/run_all.m`. Las rutinas de entrenamiento son `train_static_ctrl.m` (gradiente estático, con momento y mejores pesos) y `train_bptt_ctrl.m` (retropropagación en el tiempo vectorizada, con soporte de realimentación parcial vía la matriz C , perturbación y ruido). Cada experimento C1–C4 guarda sus figuras en `Informe/figs/` y sus métricas en `*_metrics.mat`.